

Efficient Rescue Robot Simulation in Webots (Project PDF Guide)

A technically strong, efficient, and report-ready robotics simulation project using obstacle avoidance, mapping, exploration, and planning.

1. Project Overview

This project simulates an **Autonomous Rescue Robot** in **Webots** that can explore an unknown environment, avoid obstacles, detect victims, and efficiently navigate using modern robotics algorithms. Unlike typical rescue robot projects that rely on random wandering, this project focuses on **efficiency** through mapping and systematic exploration.

2. Project Objectives

- Autonomously explore an unknown indoor environment (maze/collapsed building simulation).
- Avoid static and dynamic obstacles using local obstacle avoidance methods.
- Build a map of the environment (occupancy grid).
- Select unexplored target areas using frontier-based exploration.
- Plan shortest paths to targets using A* path planning.
- Detect victims using camera-based detection (color segmentation or object detection).
- Log victim locations and generate mission summary output.
- Minimize travel time and unnecessary repeated exploration (efficiency focus).

3. Why Webots?

Webots is a robotics simulator that provides realistic physics, sensors (LiDAR, camera, GPS, IMU), and ready-to-use robot models. It is suitable for academic projects because it allows rapid prototyping while keeping the simulation close to real-world robotics workflows.

4. System Architecture (Recommended Design)

A strong rescue robot should use a **two-layer navigation architecture**:

(A) Global Layer (Decision + Planning) Mapping (Occupancy Grid) Exploration (Frontier Selection) Path Planning (A* Algorithm) **(B) Local Layer (Real-Time Safety)** Local Obstacle Avoidance (Potential Field or simplified DWA) Emergency Stop / Recovery behaviors

5. Robot Model and Sensors (Recommended)

Component	Purpose	Recommendation
Mobile Robot Base	Movement in the world	Pioneer 3-DX or TurtleBot3

LiDAR (2D)	Accurate distance scanning + mapping	Highly recommended (major upgrade)
Camera	Victim detection	RGB camera with front-facing mount
Wheel Encoders	Odometry (movement tracking)	Built-in for most robots
GPS (optional)	Ground-truth location logging	Use if allowed by project rules
Compass/IMU	Orientation stabilization	Recommended for better localization

6. Best Algorithms to Use (Efficiency-Focused)

This section lists the most recommended algorithms to make your rescue robot technically strong and efficient.

6.1 Occupancy Grid Mapping (Core Mapping Method)

Occupancy Grid Mapping divides the environment into small grid cells and classifies each cell as:

- **Free** (no obstacle)
- **Occupied** (obstacle present)
- **Unknown** (not explored yet)

Why it improves efficiency: The robot avoids revisiting already-explored areas and can plan shortest paths using A*.

Input sensors: LiDAR (preferred) or multiple distance sensors.

6.2 Frontier-Based Exploration (Most Important Upgrade)

Frontier-Based Exploration selects the next target location based on the boundary between known and unknown space. A **frontier** is a region where explored free space touches unexplored cells.

Algorithm steps:

1. Build occupancy grid map
2. Identify frontier cells
3. Cluster frontiers into groups
4. Choose best frontier based on distance / size / safety
5. Plan a path to the selected frontier

Why it is efficient: It prevents random wandering and provides systematic coverage of the environment.

6.3 A* Path Planning (Best Global Planner)

A* (A-Star) is a shortest-path planning algorithm that uses heuristics to efficiently search the best route. It is widely used in robotics and game AI.

Why A* is recommended: Fast and easy to implement Optimal (finds shortest path if heuristic is admissible) Works perfectly with occupancy grid maps **Common heuristic:** Manhattan distance (grid) or Euclidean distance.

6.4 Local Obstacle Avoidance (Real-Time Collision Prevention)

Even with A* planning, robots need local obstacle avoidance for real-time safety. Recommended methods:

(A) Potential Field Method

- Goal generates attractive force
- Obstacles generate repulsive force
- Robot moves in direction of combined vector

(B) Dynamic Window Approach (DWA) (More advanced)

- Samples possible velocities and turning rates
- Predicts future trajectory for each option
- Chooses the safest and most goal-directed motion

Recommendation: Use Potential Field (simpler) or simplified DWA (more impressive).

6.5 Victim Detection (Computer Vision)

Victim detection can be implemented using a Webots camera. Common efficient methods:

(A) Color Segmentation (Fast + Reliable)

- Convert image to HSV
- Threshold for victim color (red/green/blue)
- Detect blobs (connected pixels)

(B) Shape Detection (Optional)

- Identify victim cylinders or specific shapes

(C) ML-Based Object Detection (Optional)

- More advanced but requires dataset/training

Recommendation: Color segmentation is best for fast and stable project results.

6.6 Path Smoothing (Efficiency Improvement)

A* paths often produce zigzag routes due to grid movement. **Path smoothing** reduces unnecessary turning and shortens travel time.

Simple method: Line-of-sight smoothing (remove intermediate waypoints if straight path is collision-free)
This small addition significantly improves efficiency and makes motion appear more realistic.

7. Performance Metrics (How to Prove Efficiency)

- Total mission completion time (seconds)
- Total distance traveled (meters)
- Number of collisions (should be 0)

- Coverage percentage of environment explored (%)
- Number of victim detections (correct vs false positives)
- Average planning time per A* run (milliseconds)
- Path efficiency ratio: (optimal path length / traveled length)

8. Experimental Comparison (Recommended for High Marks)

To demonstrate technical improvement, compare at least two methods:

Baseline: Random walk + obstacle avoidance

Proposed: Frontier exploration + occupancy grid + A*

Show results in a table (time taken, distance traveled, coverage). This proves your robot is more efficient than typical rescue robot simulations.

9. Implementation Plan (Fast Timeline)

- **Day 1-2:** Create world (maze + debris), choose robot, add LiDAR + camera
- **Day 3:** Implement obstacle avoidance (potential field / reactive)
- **Day 4:** Implement occupancy grid mapping
- **Day 5:** Implement frontier detection and selection
- **Day 6:** Implement A* planning and waypoint following
- **Day 7:** Implement victim detection + GPS logging
- **Day 8:** Add smoothing + replanning
- **Day 9-10:** Testing, evaluation, report writing, final demo video

10. High-Level Pseudocode (Report-Friendly)

Main Loop:

1. Read LiDAR / distance sensors
2. Update occupancy grid map
3. If victim detected by camera:
 - Stop, log GPS position, mark victim as found
4. If no current target OR target reached:
 - Compute frontier cells
 - Select best frontier (nearest or largest)
 - Run A* to generate path
 - Smooth path waypoints
5. Follow planned waypoints
6. Apply local obstacle avoidance if obstacle appears unexpectedly
7. Repeat until all victims found or exploration complete

11. Final Deliverables Checklist

- Webots world (.wbt) file with maze/disaster environment
- Robot controller code (Python/C) implementing full algorithm pipeline
- Victim detection output (console logs + coordinates)
- Performance comparison results (baseline vs improved)
- Final report (problem, methodology, results, conclusion)
- Short demo video showing navigation and victim detection

12. Recommended Best Algorithm Stack (Final Recommendation)

Best Efficiency Stack: LiDAR-based Occupancy Grid Mapping Frontier-Based Exploration (target selection) A* Global Path Planning Path smoothing (line-of-sight) Local obstacle avoidance (Potential Field or simplified DWA) Camera-based victim detection (HSV color segmentation) Replanning if blocked (dynamic obstacles) This combination is realistic, technically strong, and achievable within a short project timeline.